

BED-1304 Python-Lab: Koding og dataanalyse for økonomer

Pandas

← tema 5 i kurset.

Hva det er, og hva det gjør

Kapittel 5 — Databehandling

Handelshøgskolen ved UiT | Høsten 2026

Ingen kode til å begynne med i dag

`import pandas as pd`
↑ ↑
biblioteket "alias"

Først: **Hva** er Pandas?

`import numpy as np`
← alias for
fil numpy.
bibliotek i Python.

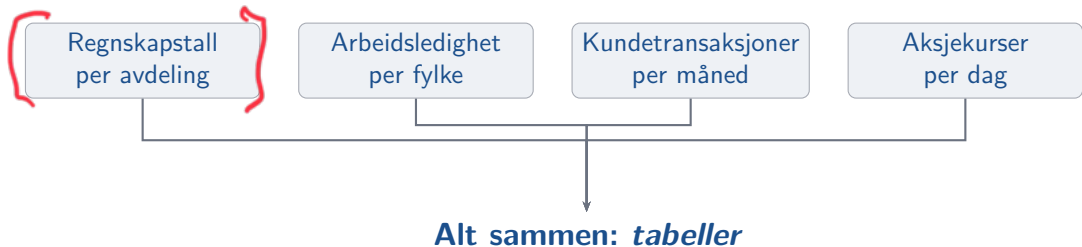
Det du skal sitte igjen med

- Hva en DataFrame er
- Hvor dataene **kommer fra**
- De **fire operasjonene** som dekker nesten alt
- Hvorfor dette er *økonomifag*, ikke bare verktøy

Syntaksen kommer etterpå

Vi åpner notatboken om litt. Da er poenget å *gjenkjenne* ideene fra denne økten i koden.

Utgangspunktet: en økonoms arbeidsdag



Rader er observasjoner. Kolonner er variabler.

Har du sett ett datasett, har du sett formen på dem alle.

Dette kjenner dere allerede

kolonner.

	↓	↓	↓	↓
	Land	BNP per innb	Utdanning	Befolkning
→	Norge	73 263	11,65	4 886 000
→	Sverige	43 001	11,60	9 378 000
→	Danmark	45 000	10,20	5 548 000
→	Polen	12 500	9,90	38 000 000

← overskrifter

← én rad = ett land

To måter å gjøre den samme jobben på



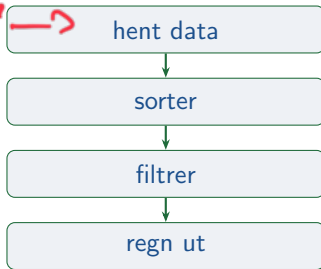
"Excel"
I regnearket



→ Hva ble egentlig gjort?
Ingen vet. Heller ikke du, om et halvår.

"Python"
I en notatbok

*internett
maskiner*



Hvert steg står skrevet ned.
Kjør på nytt — samme svar, hver gang.

Definisjonen

Pandas er et regneark du kan programmere.

Samme tabell. Men hver operasjon er en *skrevet kommando* i stedet for et museklikk — og en skrevet kommando kan gjentas, deles, granskes og rettes.

SSB, Norges Bank, Finanstilsynet, konsulenthuse og forskning
leverer i dag koden sammen med tallene. Fordi det er *sporbart*

Anatomien: tre deler, tre navn

Indeks
radenes
navn

	Land	BNP per innb	Utdanning	Betelling
0	Norge	73 263	11,65	4.700.000
1	Sverige	43 001	11,60	8.900.000
2	Danmark	45 000	10,20	5.000.000
3	Polen	12 500	9,90	20.000.000

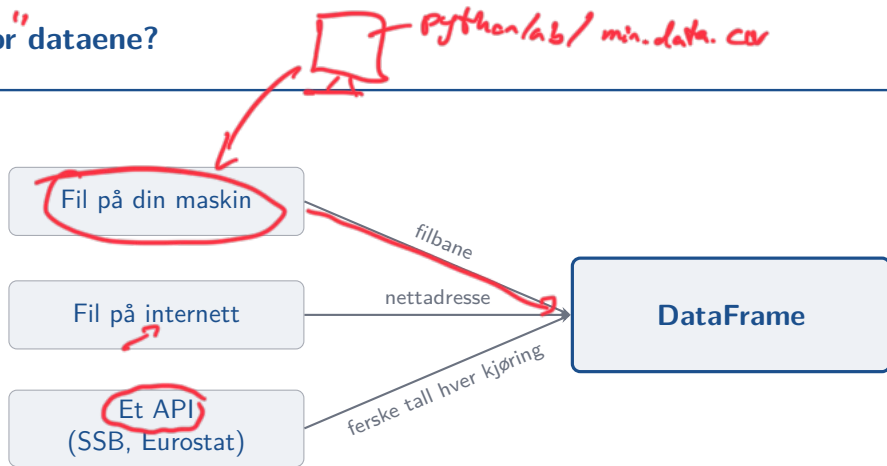
Kolonner = variabler

En kolonne = en Series

ny kolonne
BNP_per_land

Hele tabellen heter en **DataFrame**. Én kolonne av gangen heter en **Series**.

Hvor "bor" dataene?



Pandas bryr seg ikke om hvor tallene kommer fra.

Det øyeblikket dette blir interessant

Fra manuell rapport til levende dashboard

Et regneark må du **oppdatere**.

En notatbok kobla til SSBs API **oppdaterer seg selv** —
den henter de ferskeste tallene hver eneste gang du kjører den.

Arbeidsflyten — alltid den samme

"fil bone/schooling-gdp.csv"



ser rart ut? gå tilbake

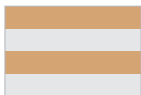
*numpy
matplotlib (stem x)
funksjoner.*

Fire verb dekker nesten alt



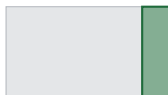
VELG

én kolonne
ut av mange



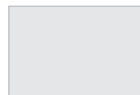
FILTRER

behold radene
som oppfyller et krav



LAG

ny kolonne regnet
ut fra de andre

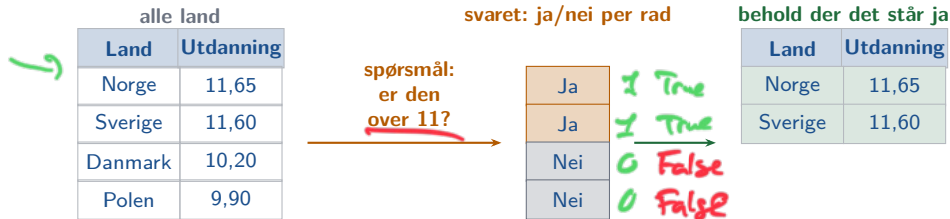


OPPSUMMER

mange tall
ned til ett

Alt annet dere møter er varianter og kombinasjoner av disse fire.

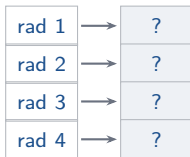
Verbet som er verdt å bruke tid på: filtrere



Filtrering er **to steg**, ikke ett:

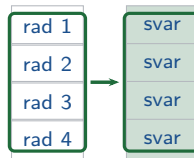
først et *ja/nei-spørsmål* til hver rad — så *behold* radene som svarte ja.

Slik dere har tenkt hittil



én rad om gangen
(en løkke)

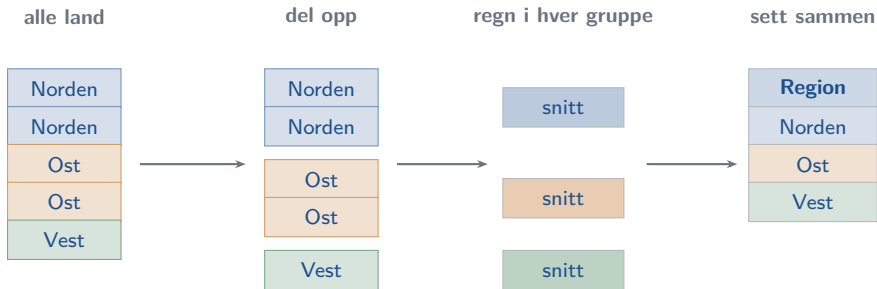
Slik Pandas gjør det



hele kolonnen
på én gang

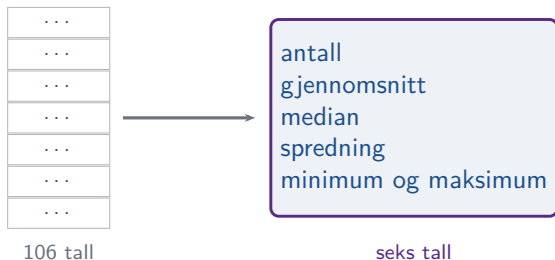
“Gang BNP med befolkning” — én setning, uansett om det er 4 rader eller 4 millioner.

Gruppering: del opp, regn, sett sammen



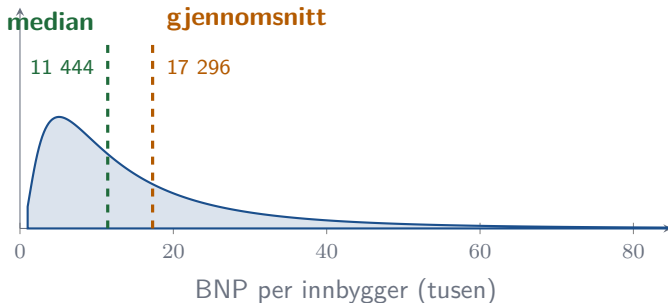
Omsetning per avdeling. Snittlønn per bransje. Ledighet per fylke.

Å oppsummere er å kaste informasjon — med vilje



Én kommando gir deg hele fordelingen i seks tall.

Og da ser du med én gang noe viktig



Gjennomsnittet ligger **langt over** medianen. Fordelingen har en lang hale mot høyre.

Noen få svært rike land drar snittet opp. Medianlandet er fattigere enn "gjennomsnittet" antyder.

Når noen viser deg et **gjennomsnitt**,
er ditt neste spørsmål alltid:
“Hva er medianen?”

Pandas gir deg begge deler samtidig. Det er hele poenget med å se på fordelingen.

Ekte data er rotete

Land	BNP	Utdanning
Norge	73 263	11,65
Finland	39 000	?
Danmark	45 000	10,20
Polen	12 500	9,90
Polen	12 500	9,90

manglende verdi

samme rad to ganger

Duplikater er stort sett en *teknisk* feil. Fjern dem.
Manglende verdier er **ikke** en teknisk feil.

Hva gjør du med hullet?



Det finnes ingen “riktig” valg her. Dette er et **faglig valg** som må begrunnes.

Samme tall, to former

BREDT — lett å lese

Land	2020	2021	2022
Norge	...	...	...
Sverige	...	...	...

Slik en avis trykker en tabell.

LANGT — lett å regne på

Land	År	Verdi
Norge	2020	...
Norge	2021	...
Norge	2022	...
Sverige	2020	...

Én observasjon per rad. Slik verktøyene vil ha det.

Pandas kan snu tabellen mellom disse to formene. Dere kommer til å trenge det — og nå vet dere i det minste hva det heter, og hvorfor.

Hva Pandas *ikke* er

Ikke et orakel

Verktøyet forteller deg *hva* tallene er. Aldri *hva de betyr*.

Ingen kvalitetskontroll

Feil i dataene inn blir pene tabeller ut. Pandas sier ikke fra.

Pandas gjør deg **raskere**. Ikke klokere.

De tre ordene

- **DataFrame** — hele tabellen
- **Series** — én kolonne
- **Indeks** — radenes navn

De fire verbene

- **Velg** en kolonne
- **Filtrer** rader på et krav
- **Lag** en ny kolonne
- **Oppsummer** til få tall

De tre påminnelsene

- Se på dataene *før* du regner
- Gjennomsnitt uten median er et halvt svar
- Å rydde i data er å ta *valg* — skriv dem ned

Nå åpner vi notatboken.

Se etter de fire verbene i koden.

De er der, hele veien.

Fra begrep til syntaks

Det vi kalte det

Det du skriver

Hele tabellen (DataFrame)

`data`

Én kolonne (Series)

`data['Utdanning']`

Velg

`data['kolonne']`

Filtrer

`data[data['kolonne'] > 11]`

Lag

`data['ny'] = data['a'] * data['b']`

Oppsummer

`data['kolonne'].mean()`

Seks linjer. Resten av kapitlet er varianter av disse.

Steg 0: hent dataene inn

Tre kilder, én funksjon

```
import pandas as pd

# 1. Fra internett
url = "https://raw.githubusercontent.com/.../schooling-gdp.csv"
data = pd.read_csv(url)

# 2. Fra fil ved siden av notatboken
data = pd.read_csv("data/schooling-gdp.csv")

# 3. Fra fil på Windows -- husk r foran anførselstegnet
data = pd.read_csv(r"C:\Users\navn\Desktop\fil.csv")
```

Den ene feilen alle får

`FileNotFoundError` betyr *alltid* at stien er feil. Bruk alternativ 1.

Steg 1: se på dataene før du regner

Kode

```
data.shape  
data.head()  
data.columns  
data.dtypes  
data.info()
```

data.shape

```
(106, 5)
```

Kjør shape *først*: ett tall forteller om du fikk inn datasettet.

data.head()

	Land	Landkode	BNP_per_capita	Utdanning	Befolkning
0	Norway	NOR	73262.680	11.65	4886000
1	Kuwait	KWT	67029.523	6.39	2992000
2	Luxembourg	LUX	57882.809	11.33	508000
3	Switzerland	CHE	55688.020	12.92	7809000
4	United States	USA	49500.629	13.24	309011008

Datatyper — og en versjonsfelle

Kode

```
data.dtypes
```

Ikke bli forvirret

Tekstkolonner heter `object` i pandas 2.x, men `str` i pandas 3.0.

Det betyr det samme. Sjekk hvilken versjon dere kjører med `pd.__version__`.

Hva du får

Land	<code>object</code>
Landkode	<code>object</code>
BNP_per_capita	<code>float64</code>
Utdanning	<code>float64</code>
Befolkning	<code>int64</code>
dtype:	<code>object</code>

`int64` = heltall | `float64` = desimaltall

Verb 1: VELG

Kode

```
# En kolonne -> Series
data['Utdanning']

# Liste av kolonner -> DataFrame
data[['Land', 'Utdanning']]

# Sjekk selv hva du fikk
type(data['Utdanning'])
type(data[['Utdanning']])
```

data['Utdanning']

```
0    11.65
1     6.39
2    11.33
3    12.92
4    13.24
Name: Utdanning, dtype: float64
```

De to typene

```
<class 'pandas.core.series.Series'>
<class 'pandas.core.frame.DataFrame'>
```

Enkle klammer → Series. Doble klammer → DataFrame.

Verb 2: FILTERER — gjør det i to steg

Steg 1: still spørsmålet

```
maske = data['Utdanning'] > 11
maske
maske.sum()
```

Steg 2: bruk svaret

```
data[maske]

# Eller alt paa en linje:
data[data['Utdanning'] > 11]
```

maske

```
0    True
1   False
2    True
3    True
4    True
Name: Utdanning, dtype: bool
```

maske.sum()

```
4
```

`maske.sum()` teller antall True — fordi True er 1 og False er 0. Akkurat som i kapittel 4.

Filtrering: legg merke til indeksen

Kode

```
data[data['Utdanning'] > 11]
```

Hva du får

	Land	Landkode	BNP_per_capita	Utdanning	Befolkning
0	Norway	NOR	73262.680	11.65	4886000
2	Luxembourg	LUX	57882.809	11.33	508000
3	Switzerland	CHE	55688.020	12.92	7809000
4	United States	USA	49500.629	13.24	309011008

Radnumrene er **0, 2, 3, 4** — ikke 0, 1, 2, 3.

Pandas beholder de opprinnelige radnavnene. Rad 1 (Kuwait) er borte, og hullet blir stående.

Det er derfor `loc` og `iloc` slutter å gi samme svar etter en filtrering.

Flere betingelser: parentesene er ikke valgfrie

Riktig

```
data[(data['Utdanning'] > 11) & (data['BNP_per_capita'] > 50000)] # OG  
data[(data['Utdanning'] > 12) | (data['Land'] == 'Norway')]      # ELLER
```

Feil — gir en kryptisk feilmelding

```
data[data['Utdanning'] > 11 & data['BNP_per_capita'] > 50000]
```

Regelen

Bruk & og | — ikke and og or.
Og sett parentes rundt *hver enkelt* betingelse.

loc = *label*

```
data.loc[0, 'Land']  
data.loc[0:4, ['Land', 'Utdanning']]  
  
# loc tar ogsaa en maske  
data.loc[data['Land'] == 'Norway']
```

iloc = *integer*

```
data.iloc[0, 0]  
data.iloc[0:5, 0:3]  
  
# iloc tar ALDRI en maske
```

Huskeregel

loc → **l**abel (navn på rad og kolonne)

iloc → **i**nteger (posisjon 0, 1, 2 ...)

På et usortert, ufiltrert datasett gir de samme svar. Etter en filtrering gjør de det ikke.

Verb 3: LAG en ny kolonne

Kode

```
data['BNP_total'] = data['BNP_per_capita'] * data['Befolkning']  
data['BNP_i_tusen'] = data['BNP_per_capita'] / 1000
```

```
data[['Land', 'BNP_total']].head()
```

	Land	BNP_total
0	Norway	3.579615e+11
1	Kuwait	2.005523e+11
2	Luxembourg	2.940447e+10
3	Switzerland	4.348677e+11
4	United States	1.529624e+13

Ingen løkke. Hele kolonnen på én gang. $3.579615e+11$ er $3,58 \cdot 10^{11}$.

Sortering

Kode

```
data.sort_values('BNP_per_capita', ascending=False).head(10)
```

`ascending=False` **synkende** — størst øverst

`ascending=True` stigende — minst øverst (dette er standard)

Merk deg dette

Sorteringen lager en **ny** tabell. Den opprinnelige data er uendret.

Vil du beholde resultatet, må du tilordne det: `sortert = data.sort_values(...)`

Verb 4: OPPSUMMER

Ett tall av gangen

```
data['BNP_per_capita'].mean()
data['BNP_per_capita'].median()
data['BNP_per_capita'].std()
data['BNP_per_capita'].min()
data['BNP_per_capita'].max()
```

Alt på én gang

```
data.describe()
```

data.describe() — utdrag

	BNP_per_capita	Utdanning
count	106.000000	106.000000
mean	17295.935035	8.629434
std	16527.233275	2.918427
min	634.971370	1.950000
50%	11444.386500	8.745000
max	73262.680000	13.240000

Snitt 17 296 mot median 11 444.

Fordelingen er høyreskjev — akkurat som vi tegnet.

Manglende verdier

Finn dem

```
data.isna()           # True/False per celle  
data.isna().sum()     # antall per kolonne
```

Gjør noe med dem

```
# Fyll med medianen  
med = data['Utdanning'].median()  
data['Utdanning'] = data['Utdanning'].fillna(  
    med)  
  
# Eller kast radene  
data = data.dropna()
```

data.isna().sum()

```
Land           0  
Landkode       0  
BNP_per_capita 0  
Utdanning      0  
dtype: int64
```

Skriv det slik — alltid

```
data['kol'] =  
data['kol'].fillna(...)  
inplace=True på en enkeltkolonne  
virker ikke.
```

Duplikater

Kode

```
data.duplicated()          # True/False per rad  
data.duplicated().sum()    # hvor mange duplikater?  
  
data = data.drop_duplicates() # tilordne resultatet!
```

Mønsteret dere har sett tre ganger nå

sort_values, drop_duplicates, dropna, fillna ...

De lager alle en **ny** tabell og lar originalen være.

Vil du beholde endringen: **tilordne resultatet til et navn.**

Gruppering: split-apply-combine i kode

Kode

```
# Snitt per gruppe
df.groupby('Region')['BNP_per_capita'].mean()

# Hvor mange i hver gruppe?
df.groupby('Region').size()

# Flere maal samtidig
df.groupby('Region')['BNP_per_capita'] \
    .agg(['mean', 'median', 'count'])
```

.mean()

```
Region
Norden      47052.64
Ost-EU      13250.00
Vest-EU     41000.00
Name: BNP_per_capita, dtype: float64
```

.agg([...])

	mean	median	count
Region			
Norden	47052.64	43000.5	5
Ost-EU	13250.00	13250.0	2
Vest-EU	41000.00	41000.0	1

Grupper på → velg kolonne → regn ut.

En rask figur — rett fra Pandas

Kode

```
data['Utdanning'].plot(kind='hist');  
  
data.plot(kind='scatter', x='Utdanning', y='BNP_per_capita');  
  
data.plot(x='maaned', y='indeks', figsize=(12, 6));    # tidsserie: bredt format
```

To små triks

- Semikolon til slutt fjerner den rare linja <Axes: ...> over figuren.
- `figsize=(bredde, høyde)` i tommer. Tidsserier vil ha *bredt* format.

Dette er grovskisser til deg selv. Pene figurer lager vi med matplotlib og seaborn senere i kurset.

Fire feilmeldinger dere kommer til å møte

Feilmelding	Hva den egentlig betyr
<code>FileNotFoundError</code>	Stien er feil. Alltid.
<code>KeyError:</code> <code>'Utdaning'</code>	Kolonnenavnet finnes ikke — skrivefeil eller feil stor bokstav. Kjør <code>data.columns</code> og kopier.
Feil ved <code>&</code>	Du glemte parentes rundt hver betingelse.
Ingen feil, men ingenting endret seg	Du glemte å tilordne resultatet.

Les feilmeldingen **nederst** først.

Se på dataene

```
data.shape
data.head()      data.tail()
data.info()      data.columns
data.dtypes      data.describe()
```

Endre

```
data['ny'] = data['a'] * data['b']
data.sort_values('kol', ascending=False)
data = data.drop_duplicates()
data['kol'] = data['kol'].fillna(0)
```

Velg og filtrer

```
data['kol']
data[['kol1', 'kol2']]
data[data['kol'] > 11]
data.loc[0, 'kol']
data.iloc[0, 0]
```

Oppsummer

```
data['kol'].mean()
data['kol'].median()
data['kol'].value_counts()
data.groupby('gr')['kol'].mean()
```

Alt annet finner dere på pandas.pydata.org/docs — eller ved å google feilmeldingen.